SVKM's NMIMS

Mukesh Patel School of Technology Management & Engineering (Mumbai Campus)

Computer Engineering Department (Diploma Sem IV)

Java Programming

Mini Project

PART A

(Part A: TO BE REFFERED BY STUDENTS)

A.1 AIM:

Develop a Mini Project in Java

A.2 Pre requisite:

Basic Knowledge of Java & Collections.

A.3 Outcome:

After successful completion of this experiment students will be able to:

1. Develop a standalone application using Java

A.4 Theory:

A mini project is desirable to be completed by a group of two or three students that cover following tools.

Java Basics

Collections

A.5 Procedure/Task:

1. Develop an application using Java
2. Prepare the document. Save and close the file and name it as RollNo._MiniProject.

SVKM's NMIMS

Mukesh Patel School of Technology Management & Engineering (Mumbai Campus)

Computer Engineering Department (Diploma Sem IV)

Java Programming

Mini Project

PART B

(PART B: TO BE COMPLETED BY STUDENTS)

(Students must submit the soft copy as per following segments within two hours of the practical. The soft copy must be uploaded on the Blackboard or emailed to the concerned lab in charge faculties at the end of the practical in case the there is no Black board access available)

Roll No. : A002	Name: Brijraj Singh Bhati
Class : Diploma Year 2 Semester IV	Batch : D2
Date of Experiment :	Date/Time of Submission :
Grade :	

B.1 Project Details

(Detailed description of project. It should include answers to following questions)

1. Motivation to take the proposed project

The primary motivation behind developing this Terminal Task Manager was to explore and implement modern logging systems and ANSI color codes in a Java CLI environment. While traditional console applications rely on plain text output, we wanted to experiment with creating visually appealing terminal interfaces using ANSI escape codes for colors, text formatting, and cursor control. This project provided an opportunity to build a custom Logger utility (util/Logger.java) that demonstrates:

- Color-coded log messages (success in green, errors in red, warnings in yellow, info in blue)
- Visual UI elements like bordered boxes for task display
- Screen clearing and section headers using escape sequences
- Typing animation effects for enhanced user experience
- Unicode symbols (✔, ✖, ⚠, ℹ, ⚙) for intuitive iconography

Beyond the visual aspects, the project serves as a comprehensive demonstration of core Java concepts learned during the semester, including Object-Oriented Programming, Collections Framework, Exception Handling, and String manipulation. The goal was to create something functional yet visually distinct from typical student projects.

1. Working of project

The Terminal Task Manager operates as a command-line application with user authentication and role-based access control:

Authentication Flow:

- Users can log in with predefined credentials (user1/pass123, user2/pass456, admin/admin123)
- New accounts can be created during runtime
- Three failed login attempts lock the user out

User Roles:

- Regular Users: Can create, view, search, and mark their own tasks as complete

- Admin Users: Extended permissions including deleting any task, managing users, and accessing system settings

Task Management:

- Tasks are stored in an ArrayList with auto-incrementing IDs
- Each task contains: ID, title, description, priority (HIGH/MEDIUM/LOW), and completion status
- Tasks can be viewed in styled boxes using the Logger utility
- Search functionality allows finding tasks by keyword in titles

The application runs in a continuous loop, displaying menus and processing user input until the exit option is selected. All user interactions are enhanced with visual feedback through the custom Logger class.

1. Advantages & disadvantages of project

Advantages:

- Clean modular architecture with packages (model, service, exception, main, util)
- Custom Logger class provides professional-looking terminal output with colors and formatting
- Role-based access control demonstrates inheritance (AdminUser extends User)
- Comprehensive exception handling with custom exceptions (TaskNotFoundException, InvalidTaskException)
- Uses multiple Java Collections: ArrayList for tasks, LinkedList concept for history
- Iterator pattern used for task traversal and deletion
- StringBuffer-like operations in Logger for building formatted output
- Self-contained application requiring no external dependencies beyond Java JDK
- Visual feedback improves user experience compared to plain console apps

Disadvantages:

- No persistent storage - all data is lost when the application exits
- In-memory user storage with plaintext passwords (not secure for production)
- Limited to terminal environments that support ANSI escape codes
- No multi-threading implementation for concurrent operations
- User interface is text-based only - no GUI option available
- Task assignments are tracked but not fully integrated with the task lifecycle
- Admin user management features are basic (activate/deactivate only)

1. Technologies used for developing the project

Core Technologies:

- Java Development Kit (JDK) 8+ - Core programming language and runtime
- Object-Oriented Programming (OOP) - Classes, Objects, Encapsulation, Inheritance
- Java Collections Framework - ArrayList, List, Iterator
- Exception Handling - try-catch blocks, custom exceptions
- String Handling - String manipulation, StringBuilder concepts
- ANSI Escape Codes - Terminal color and formatting control

Development Tools:

- Standard Java compiler (javac)
- Command-line terminal for execution
- Git for version control

Package Structure:

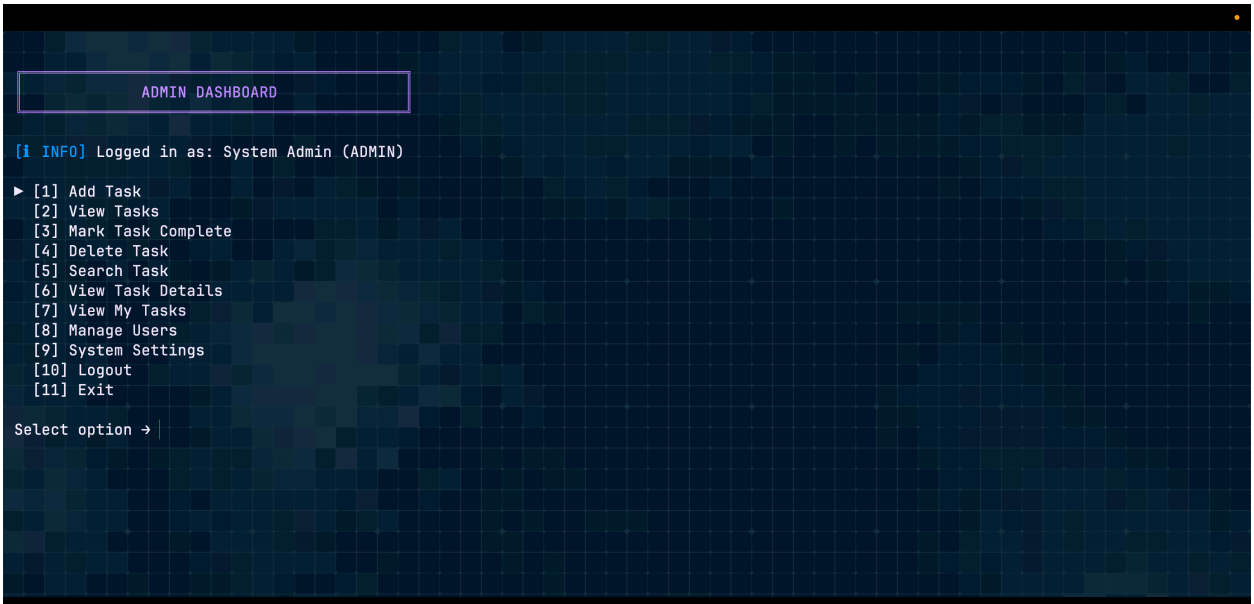
- model package: Task, User, AdminUser classes
- service package: TaskManager (business logic)
- exception package: Custom exception classes
- util package: Logger (ANSI color handling and UI)

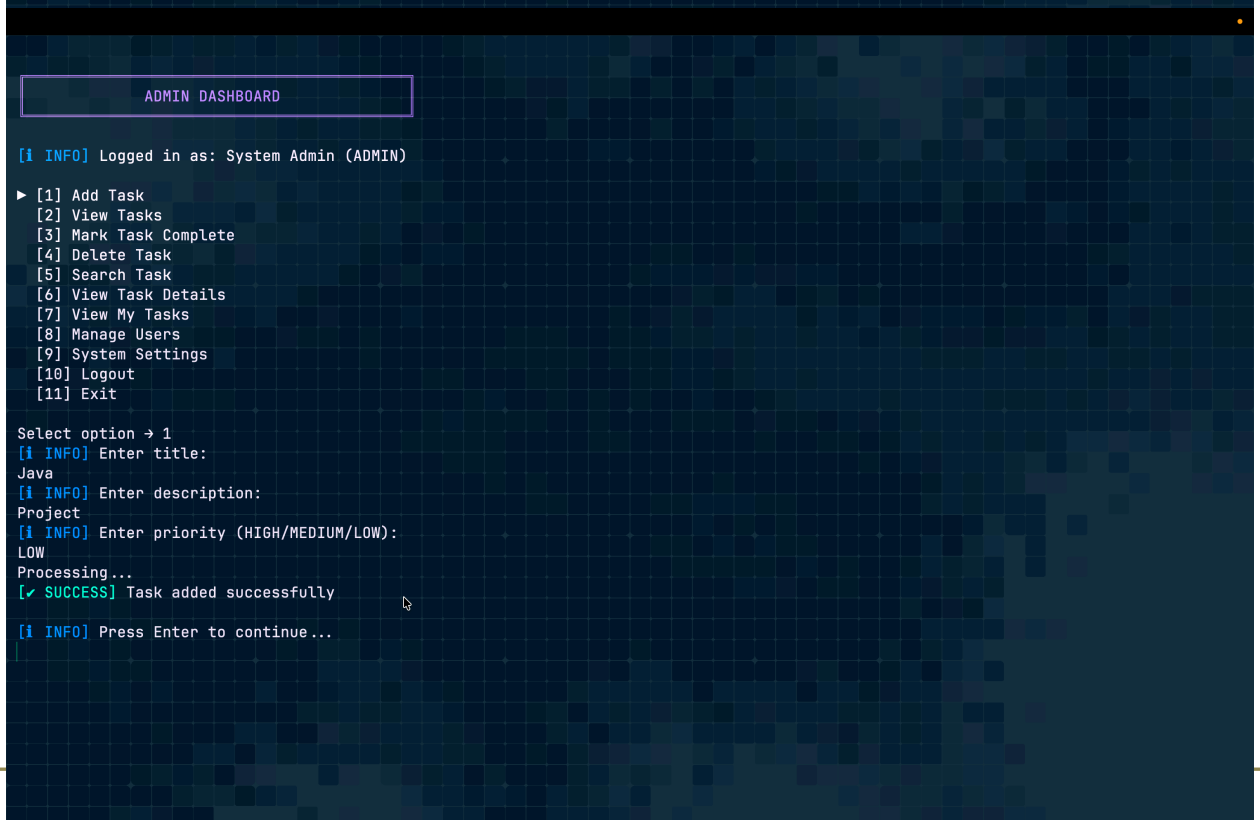
- main package: MainApplication (entry point)

Key Features Implemented:

- ANSI color codes for terminal styling
- Unicode symbols for visual indicators
- Screen clearing escape sequences
- Box-drawing characters for UI elements
- Role-based permission system

B.2 Output





B.3 Project Code :

The source code and folder structures are too long to be pasted as plain text.

The GitHub Repository has been attached below, please consider:

https://github.com/laz4rd/Task-Manager-Project

B.4 Conclusion:

The Terminal Task Manager successfully fulfills its purpose as a semester-end project demonstrating core Java programming concepts. The application goes beyond basic requirements by implementing:

1. **A complete user authentication system** with role-based access control
2. **An aesthetically pleasing CLI interface** using ANSI colors and box-drawing characters
3. **Robust error handling** through custom exceptions and input validation
4. **Clean architecture** with proper package organization and separation of concerns

Project Strengths

1. **Visual Appeal**: The Logger utility transforms a basic terminal application into an engaging user experience with colors, icons, and structured formatting.
2. **Security Awareness**: While not production-grade, the implementation shows understanding of authentication concepts including password validation, account status, and permission checking.
3. **Code Organization**: The package structure and class responsibilities demonstrate professional software development practices.
4. **Complete Feature Set**: All CRUD operations are implemented, plus search functionality and detailed viewing.

Potential Enhancements

For future development, the following improvements would add significant value:

1. **Data Persistence**: Implement file-based storage (JSON, XML, or database) to retain tasks between sessions
2. **Password Security**: Add hashing (bcrypt/SHA-256) for password storage
3. **Task Assignment**: Complete the `assignedTasks` functionality to actually assign tasks to users
4. **Task Editing**: Add ability to modify existing tasks

5. **Sorting and Filtering**: Implement priority-based sorting and status filtering
6. **Unit Testing**: Add JUnit tests for service layer methods
7. **Configuration**: Externalize user credentials and application settings

Team Members:

- A002 - Brijraj Singh Bhati (Lead Developer & System Architect)
- A015 - Khushi Mendon (Core Logic Engineer)
- A041 - Manan Parmar (Interface & CLI Engineer)

B.5 Observations and Learning:

1. Architecture and Design

Package Organization The project follows a well-structured package hierarchy:

- `main/` - Application entry point and UI flow control
- `model/` - Data entities (Task, User, AdminUser)
- `service/` - Business logic (TaskManager)
- `util/` - Helper utilities (Logger, ANSI_Test)
- `exception/` - Custom exception classes

This separation of concerns makes the codebase maintainable and aligns with industry-standard Java project structures.

Inheritance Implementation The `AdminUser` class properly extends `User`, demonstrating inheritance concepts:

- AdminUser inherits all User attributes and methods
- Overrides `hasPermission()` to provide extended privileges
- Adds admin-specific methods like `canManageUsers()`, `canDeleteAnyTask()`
- Maintains a permission list with granular access control

2. Exception Handling

The project implements custom exceptions for specific error scenarios:

- `InvalidTaskException` - Thrown when task validation fails (empty title, invalid priority)
- `TaskNotFoundException` - Thrown when attempting to access non-existent tasks

This approach provides:

- Clear error messages to users
- Type-safe error handling with try-catch blocks
- Better debugging and error tracing
- Separation of business logic from error handling

3. Collections Framework Usage

ArrayList for Task Storage The `TaskManager` uses `ArrayList<Task>` for dynamic task storage, providing:

- O(1) access by index
- Dynamic resizing
- Easy iteration with enhanced for-loops
- Iterator support for safe deletion during traversal

User Collection The `MainApplication` maintains a `List<User>` for user management, enabling:

- User registration and authentication
- Role-based access control
- User activation/deactivation

4. User Authentication and Security

The authentication system includes:

- Username/password validation
- Account activation status checking
- Maximum login attempt limiting (3 attempts)
- Role-based permission checking

Permission System Both User and AdminUser implement a permission system:

- Regular users have basic task creation/viewing permissions
- Admins have extended permissions for user management and task deletion
- Method-level permission checks before executing sensitive operations

5. Terminal UI Enhancement

Logger Utility Class The `Logger` class significantly enhances user experience:

- ANSI color codes for visually appealing output (success=green, error=red, etc.)
- Section headers with box-drawing characters (╔═══╗ style)
- Task boxes with formatted display showing ID, title, priority, and status
- Typing effect simulation with configurable delay
- Screen clearing capability using escape sequences

- Consistent icon usage (✔, ✖, ⚠, ℹ, ⚙) for different message types

Input Validation The application handles invalid inputs gracefully:

- Try-catch blocks around Scanner operations
- Error messages for non-numeric menu selections
- Invalid ID format handling
- Empty input validation

6. Code Quality Observations

Strengths:

- Consistent naming conventions throughout
- Proper encapsulation with private fields and public getters/setters
- Defensive copies returned from collections (`new ArrayList<>(assignedTasks)`)
- String validation using regex for priority levels
- Text truncation to prevent layout breaking in UI

Areas for Improvement:

- Some Hungarian notation remnants in Task class (`_titleString`, `_descriptionString`)
- Passwords stored in plain text (not hashed)
- No data persistence - all data lost on application exit
- No actual task assignment logic despite having `assignedTasks` list
- Some unused methods (`viewAllTasksAdmin()` is defined but not called)
- Hardcoded demo user credentials visible in source

7. Educational Value

This project successfully demonstrates:

- **OOP Concepts**: Classes, objects, inheritance, encapsulation
- **Collections**: ArrayList usage and Iterator pattern
- **Exception Handling**: Custom exceptions and try-catch blocks
- **String Manipulation**: Regex validation, formatting, truncation
- **Package Organization**: Proper modular structure
- **User Input Handling**: Scanner usage with validation
- **UI/UX in CLI**: ANSI escape sequences for enhanced visuals

Technical Learning Outcomes

Understanding of Java Fundamentals

1. **Class Design**: The Task, User, and AdminUser classes demonstrate proper class design with:
   - Private fields for encapsulation
   - Constructor overloading
   - Getter and setter methods
   - Custom `toString()` implementations
2. **Inheritance**: AdminUser extends User, showing:
   - `super()` constructor calls
   - Method overriding
   - Polymorphism through permission checking
3. **Static vs Instance**: Logger uses static methods appropriately since it doesn't maintain state, while TaskManager uses instance methods for task list management.

Collections Proficiency

The project demonstrates practical use of:

- `ArrayList` for ordered collections
- `Iterator` for safe removal during iteration
- `List` interface for abstraction
- Enhanced for-loops for traversal

Exception Handling Patterns

Custom exceptions extend `Exception` rather than `RuntimeException`, enforcing:

- Checked exceptions that must be handled
- Explicit try-catch blocks in calling code
- Meaningful error messages

Comparison with Requirements

Required Concept	Implementation Status	Location
Classes and Objects	Complete	Task, User, AdminUser
Methods and Constructors	Complete	All model classes
Inheritance	Complete	AdminUser extends User

SVKM's NMIMS

Mukesh Patel School of Technology Management & Engineering (Mumbai Campus)

Computer Engineering Department (Diploma Sem IV)

Java Programming

Mini Project

Packages	Complete	main, model, service, exception, util
Exception Handling	Complete	try-catch blocks, custom exceptions
String Handling	Complete	Validation, formatting, searching
Collections Framework	Complete	ArrayList usage
Iterator	Complete	TaskManager.deleteTask()

All syllabus requirements have been successfully implemented.